

Lecture 04 - Geometric Projection

This lecture explains how a 3D view becomes a 2D image. Projection is where camera geometry, homogeneous coordinates, clipping planes, depth precision, and viewport mapping meet.

Source chunk	course_text_parts/03_lectures/04_geometric-projection.txt
Extracted slide pages	61
Reader purpose	Commentary and causal explanation for human reading

How to read this lecture

- Read the section commentary before looking at the raw slide bullets.
- Use the mental model as the short version you should remember.
- Answer the check question without notes before continuing.

Per-slide commentary

Each extracted slide page gets its own note. Use this section as the spoken commentary that the terse slide deck is missing.

Page 1	Visual or title slide
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Visual or title slide' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Visual or title slide. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.
Check yourself	Can you turn 'Visual or title slide' into a causal sentence instead of repeating the slide title?
Page 2	Projecting 3D Models
Source cue	- To compute the 2D representation of a 3D model on the screen, / it needs to be projected, whereby it loses its z-coordinates / - Depending on the projection type, the object might get skewed / (perspective shortening) / - Projection can also be expressed as a matrix multiplication,

Professor-style explanation	The slide 'Projecting 3D Models' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. The terms describe model representation before rendering: - To compute the 2D representation of a 3D model on the screen, / it needs to be projected, whereby it loses its z-coordinates / - Depending on the projection type, the object might get skewed / (perspective shortening) / - Projection can also be expressed as a matrix multiplication,. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors.
Technical commentary	This slide is about Projecting 3D Models. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. The terms describe model representation before rendering: - To compute the 2D representation of a 3D model on the screen, / it needs to be projected, whereby it loses its z-coordinates / - Depending on the projection type, the object might get skewed / (perspective shortening) / - Projection can also be expressed as a matrix multiplication,. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation.
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Check yourself	Can you name the geometric objects and attributes represented by 'Projecting 3D Models' before rendering begins?

Page 3	4.1 Linear Perspective
Source cue	4.2 Planar Projections / 4.3 Camera Modeling / 4.4 Specifying Projections in OpenGL / 4.5 Perspective Projection Derivation / 4.6 Orthographic Projection Derivation
Professor-style explanation	The slide '4.1 Linear Perspective' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: 4.2 Planar Projections / 4.3 Camera Modeling / 4.4 Specifying Projections in OpenGL / 4.5 Perspective Projection Derivation / 4.6 Orthographic Projection Derivation. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.

Technical commentary	This slide is about 4.1 Linear Perspective. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: 4.2 Planar Projections / 4.3 Camera Modeling / 4.4 Specifying Projections in OpenGL / 4.5 Perspective Projection Derivation / 4.6 Orthographic Projection Derivation. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in '4.1 Linear Perspective'?

Page 4	4.1 Linear Perspective
Source cue	Discovery of perspective distortion in art
Professor-style explanation	The slide '4.1 Linear Perspective' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: Discovery of perspective distortion in art. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about 4.1 Linear Perspective. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: Discovery of perspective distortion in art. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how '4.1 Linear Perspective' changes positions before rasterization?

Page 5	Linear Perspective
Source cue	- What is wrong in this image? / William Hogarth: / False Perspective, 1754 / Satire on False Perspective / William Hogarth (1753)

Professor-style explanation	The slide 'Linear Perspective' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: - What is wrong in this image? / William Hogarth: / False Perspective, 1754 / Satire on False Perspective / William Hogarth (1753). View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Linear Perspective. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: - What is wrong in this image? / William Hogarth: / False Perspective, 1754 / Satire on False Perspective / William Hogarth (1753). View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Linear Perspective' changes positions before rasterization?
Page 6	Drawing by Projecting
Source cue	- Projection: method for imaging objects using parallel or central beams / - Perspective: / "The science which teaches how to represent tri-dimensional objects / on a bi-dimensional surface, so that the perspective image coincides with / the one which is given by direct vision."

Professor-style explanation	The slide 'Drawing by Projecting' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: - Projection: method for imaging objects using parallel or central beams / - Perspective: / "The science which teaches how to represent tri-dimensional objects / on a bi-dimensional surface, so that the perspective image coincides with / the one which is given by direct vision.". View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Drawing by Projecting. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: - Projection: method for imaging objects using parallel or central beams / - Perspective: / "The science which teaches how to represent tri-dimensional objects / on a bi-dimensional surface, so that the perspective image coincides with / the one which is given by direct vision.". View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Drawing by Projecting' changes positions before rasterization?
Page 7	Projection in the Arts 1/5
Source cue	- Before the Renaissance, pictures often had a religious / and symbolic character / - Perspective was not understood: / accordingly, they seem 2-dimensional / Fresco from an Egyptian grave

Professor-style explanation	The slide 'Projection in the Arts 1/5' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: - Before the Renaissance, pictures often had a religious / and symbolic character - Perspective was not understood: / accordingly, they seem 2-dimensional / Fresco from an Egyptian grave. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Projection in the Arts 1/5. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: - Before the Renaissance, pictures often had a religious / and symbolic character - Perspective was not understood: / accordingly, they seem 2-dimensional / Fresco from an Egyptian grave. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Projection in the Arts 1/5' changes positions before rasterization?

Page 8	Projection in the Arts 2/5
Source cue	- Techniques to simulate space: / - Shaded round volumetric shapes / - Spatial depth through converging lines / Giotto di Bondone / (1266-1337, Florence)
Professor-style explanation	The slide 'Projection in the Arts 2/5' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: - Techniques to simulate space: / - Shaded round volumetric shapes / - Spatial depth through converging lines / Giotto di Bondone / (1266-1337, Florence). Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.

Technical commentary	This slide is about Projection in the Arts 2/5. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: - Techniques to simulate space: / - Shaded round volumetric shapes / - Spatial depth through converging lines / Giotto di Bondone / (1266-1337, Florence). Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Projection in the Arts 2/5' works per object, per image region, per ray, or per fragment?

Page 9	Projection in the Arts 3/5
Source cue	- Renaissance (1400-1700) / - New emphasis on the importance of personal views and interpretation of / the world, power of observation, especially of nature / (astronomy, anatomy, botany, ...) / - Universe as a clockwork:
Professor-style explanation	The slide 'Projection in the Arts 3/5' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: - Renaissance (1400-1700) / - New emphasis on the importance of personal views and interpretation of / the world, power of observation, especially of nature / (astronomy, anatomy, botany, ...) / - Universe as a clockwork. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Projection in the Arts 3/5. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: - Renaissance (1400-1700) / - New emphasis on the importance of personal views and interpretation of / the world, power of observation, especially of nature / (astronomy, anatomy, botany, ...) / - Universe as a clockwork. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Projection in the Arts 3/5' changes positions before rasterization?

Page 10	Projection in the Arts 4/5
---------	----------------------------

Source cue	- Examples from the Renaissance
Professor-style explanation	The slide 'Projection in the Arts 4/5' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: - Examples from the Renaissance. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Projection in the Arts 4/5. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: - Examples from the Renaissance. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Projection in the Arts 4/5' changes positions before rasterization?
Page 11	Projection in the Arts 4/5
Source cue	- Examples from the Renaissance / The Last Supper / Leonardo da Vinci (1498)
Professor-style explanation	The slide 'Projection in the Arts 4/5' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: - Examples from the Renaissance / The Last Supper / Leonardo da Vinci (1498). View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.

Technical commentary	This slide is about Projection in the Arts 4/5. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: - Examples from the Renaissance / The Last Supper / Leonardo da Vinci (1498). View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Projection in the Arts 4/5' changes positions before rasterization?

Page 12	Projection in the Arts 5/5
Source cue	- "The Invention of the Painting" (1830) by Karl Friedrich Schinkel (1781-1841): / - An image based on a legend by Pliny the Elder / - Tracing the shadow contours on a drawing surface
Professor-style explanation	The slide 'Projection in the Arts 5/5' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: - "The Invention of the Painting" (1830) by Karl Friedrich Schinkel (1781-1841): / - An image based on a legend by Pliny the Elder / - Tracing the shadow contours on a drawing surface. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Projection in the Arts 5/5. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: - "The Invention of the Painting" (1830) by Karl Friedrich Schinkel (1781-1841): / - An image based on a legend by Pliny the Elder / - Tracing the shadow contours on a drawing surface. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Projection in the Arts 5/5' changes positions before rasterization?

Page 13	Projection in the Arts 6/5
Source cue	- M. C. Escher (1898 - 1972)

Professor-style explanation	The slide 'Projection in the Arts 6/5' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: - M. C. Escher (1898 - 1972). View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Projection in the Arts 6/5. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: - M. C. Escher (1898 - 1972). View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Projection in the Arts 6/5' changes positions before rasterization?

Page 14	Projection Technics 1/3
Source cue	- Filippo Brunelleschi (1377-1446): / Systematic method for obtaining / a perspective projection
Professor-style explanation	The slide 'Projection Technics 1/3' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: - Filippo Brunelleschi (1377-1446): / Systematic method for obtaining / a perspective projection. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.

Technical commentary	This slide is about Projection Technics 1/3. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: - Filippo Brunelleschi (1377-1446): / Systematic method for obtaining / a perspective projection. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Projection Technics 1/3' changes positions before rasterization?

Page 15	Projection Technics 1/3
Source cue	The Delivery of the Keys / Perugino (1482)
Professor-style explanation	The slide 'Projection Technics 1/3' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: The Delivery of the Keys / Perugino (1482). View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Projection Technics 1/3. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: The Delivery of the Keys / Perugino (1482). View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Projection Technics 1/3' changes positions before rasterization?

Page 16	Projection Technics 2/3
Source cue	- Formal description of the perspective projection / - Leone Batista Alberti (1404-1472): / First essay on the subject of "La Pittura" (1435) / - "A painting [the projection plane] is the intersection of a visual pyramid [view / frustum] at a given distance, with a fixed center [center of projection] and a

Professor-style explanation	The slide 'Projection Technics 2/3' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: - Formal description of the perspective projection / - Leone Batista Alberti (1404-1472): / First essay on the subject of "La Pittura" (1435) / - "A painting [the projection plane] is the intersection of a visual pyramid [view / frustum] at a given distance, with a fixed center [center of projection] and a. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Projection Technics 2/3. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: - Formal description of the perspective projection / - Leone Batista Alberti (1404-1472): / First essay on the subject of "La Pittura" (1435) / - "A painting [the projection plane] is the intersection of a visual pyramid [view / frustum] at a given distance, with a fixed center [center of projection] and a. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Projection Technics 2/3' changes positions before rasterization?
Page 17	Projection Technics 3/3
Source cue	- Albrecht Dürer (1471-1528): / Geometric and mechanical / description of the concept / of similar triangles / - Construction of several

Professor-style explanation	The slide 'Projection Technics 3/3' explains the moment where continuous geometry becomes a discrete image problem. A mathematical line or triangle is not made of pixels, but the screen is a grid of samples. Rasterization decides which samples are covered by the projected primitive and creates fragments for those samples. At the same time, values attached to the primitive, such as depth, color, normals, or texture coordinates, are interpolated so that each fragment has the data needed for shading. The terms describe sample generation: - Albrecht Dürer (1471-1528): / Geometric and mechanical / description of the concept / of similar triangles / - Construction of several. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments. The important distinction is that rasterization creates candidates. A fragment exists because a primitive covered a sample, but final visibility still depends on depth testing, stencil testing, blending, masking, and framebuffer operations.
Technical commentary	This slide is about Projection Technics 3/3. The rasterization objects on the slide convert ideal geometric primitives into covered samples or fragments with interpolated attributes. The terms describe sample generation: - Albrecht Dürer (1471-1528): / Geometric and mechanical / description of the concept / of similar triangles / - Construction of several. A continuous primitive is tested against the pixel grid, covered samples become fragments, and interpolated attributes travel with those fragments.
Why it matters	Rasterization determines fragment generation; without it, shading and fragment tests have nothing to operate on.
Check yourself	Can you explain which samples/fragments are generated by 'Projection Technics 3/3'?

Page 18	Projection Technics 3/3
Source cue	- Camera Obscura
Professor-style explanation	The slide 'Projection Technics 3/3' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: - Camera Obscura. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.

Technical commentary	This slide is about Projection Technics 3/3. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: - Camera Obscura. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Projection Technics 3/3' changes positions before rasterization?

Page 19	4.2 Planar Projections
Source cue	Perspective and parallel projections
Professor-style explanation	The slide '4.2 Planar Projections' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: Perspective and parallel projections. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about 4.2 Planar Projections. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: Perspective and parallel projections. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how '4.2 Planar Projections' changes positions before rasterization?

Page 20	Geometric Projections
Source cue	- Method of mapping objects onto a screen using projectors / - Planar geometric projections / - Projection beams are straight lines / - Projection surface is a plane / (image plane, picture plane,

Professor-style explanation	The slide 'Geometric Projections' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: - Method of mapping objects onto a screen using projectors - Planar geometric projections - Projection beams are straight lines - Projection surface is a plane (image plane, picture plane, . View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Geometric Projections. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: - Method of mapping objects onto a screen using projectors - Planar geometric projections - Projection beams are straight lines - Projection surface is a plane (image plane, picture plane, . View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Geometric Projections' changes positions before rasterization?
Page 21	Projection Types 1/2
Source cue	- Perspective projection (= central projection) - Established by projection center / (COP, center of projection) - Perspective reduction: The size of the projection / of an object decreases with increasing distance

Professor-style explanation	The slide 'Projection Types 1/2' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: - Perspective projection (= central projection) / - Established by projection center / (COP, center of projection) / - Perspective reduction: The size of the projection / of an object decreases with increasing distance. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Projection Types 1/2. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: - Perspective projection (= central projection) / - Established by projection center / (COP, center of projection) / - Perspective reduction: The size of the projection / of an object decreases with increasing distance. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Projection Types 1/2' changes positions before rasterization?
Page 22	Projection Types 2/2
Source cue	Planar geometric Projections / Parallel Perspective / Orthographic Oblique 1-Point / 2-Point / Top Axonometric

Professor-style explanation	The slide 'Projection Types 2/2' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: Planar geometric Projections / Parallel Perspective / Orthographic Oblique 1-Point / 2-Point / Top Axonometric. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Projection Types 2/2. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: Planar geometric Projections / Parallel Perspective / Orthographic Oblique 1-Point / 2-Point / Top Axonometric. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Projection Types 2/2' changes positions before rasterization?

Page 23	Orthographic Projections
Source cue	- Orthographic projections often used for / construction drawings of machines and parts / - Main views: Front view, top view and side view / - Advantages / - Accurate length and angle measurements possible
Professor-style explanation	The slide 'Orthographic Projections' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: - Orthographic projections often used for / construction drawings of machines and parts / - Main views: Front view, top view and side view / - Advantages / - Accurate length and angle measurements possible. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.

Technical commentary	This slide is about Orthographic Projections. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: - Orthographic projections often used for / construction drawings of machines and parts / - Main views: Front view, top view and side view / - Advantages / - Accurate length and angle measurements possible. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Orthographic Projections' changes positions before rasterization?

Page 24	Axonometric Projections
Source cue	- Projection beams perpendicular to the projection plane / Isometric / - Projection plane not perpendicular to a coordinate axis / - Visible effects / - Several perpendicular surfaces visible
Professor-style explanation	The slide 'Axonometric Projections' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Projection beams perpendicular to the projection plane / Isometric / - Projection plane not perpendicular to a coordinate axis / - Visible effects / - Several perpendicular surfaces visible. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Axonometric Projections. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Projection beams perpendicular to the projection plane / Isometric / - Projection plane not perpendicular to a coordinate axis / - Visible effects / - Several perpendicular surfaces visible. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Axonometric Projections'?
Page 25	Isometric Projections

Source cue	- Applications in illustrations, furniture design, ... / - Advantages / - Communicate spatial impression in a single view / - Same length ratios along the main axes / - Disadvantages
Professor-style explanation	The slide 'Isometric Projections' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: - Applications in illustrations, furniture design, ... / - Advantages / - Communicate spatial impression in a single view / - Same length ratios along the main axes / - Disadvantages. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Isometric Projections. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: - Applications in illustrations, furniture design, ... / - Advantages / - Communicate spatial impression in a single view / - Same length ratios along the main axes / - Disadvantages. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Isometric Projections' changes positions before rasterization?
Page 26	Oblique Projection 1/2
Source cue	- Projection rays not perpendicular to the projection plane / - Common: $\alpha = 30^\circ$ / - Cavalier projection $\alpha = 45^\circ$, i.e., $\tan(\alpha) = 1$ / a' projectors

Professor-style explanation	The slide 'Oblique Projection 1/2' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: - Projection rays not perpendicular to the projection plane / - Common: $\alpha = 30^\circ$ / - Cavalier projection $\alpha = 45^\circ$, i.e., $\tan(\alpha) = 1$ / a' projectors. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about Oblique Projection 1/2. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: - Projection rays not perpendicular to the projection plane / - Common: $\alpha = 30^\circ$ / - Cavalier projection $\alpha = 45^\circ$, i.e., $\tan(\alpha) = 1$ / a' projectors. Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'Oblique Projection 1/2' works per object, per image region, per ray, or per fragment?
Page 27	Oblique Projection 2/2
Source cue	- Advantages / - Can accurately represent the shape of a boundary surface / of an object for accurate measurements / - Missing foreshortening simplifies comparison of sizes / - Gives a good impression of the 3D appearance
Professor-style explanation	The slide 'Oblique Projection 2/2' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: - Advantages / - Can accurately represent the shape of a boundary surface / of an object for accurate measurements / - Missing foreshortening simplifies comparison of sizes / - Gives a good impression of the 3D appearance. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.

Technical commentary	This slide is about Oblique Projection 2/2. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: - Advantages / - Can accurately represent the shape of a boundary surface / of an object for accurate measurements / - Missing foreshortening simplifies comparison of sizes / - Gives a good impression of the 3D appearance. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Oblique Projection 2/2' changes positions before rasterization?

Page 28	Properties of Parallel Projections
Source cue	- Assumption: Object surface of greatest interest lies parallel to a main plane / - Direction of projection: DOP (direction of projection) / - Normal of the projection plane: VPN (view plane normal) / - Top view (orthographic) / - VPN Main axis, DOP VPN
Professor-style explanation	The slide 'Properties of Parallel Projections' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: - Assumption: Object surface of greatest interest lies parallel to a main plane / - Direction of projection: DOP (direction of projection) / - Normal of the projection plane: VPN (view plane normal) / - Top view (orthographic) / - VPN Main axis, DOP VPN. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Properties of Parallel Projections. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: - Assumption: Object surface of greatest interest lies parallel to a main plane / - Direction of projection: DOP (direction of projection) / - Normal of the projection plane: VPN (view plane normal) / - Top view (orthographic) / - VPN Main axis, DOP VPN. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.

Check yourself	Can you explain how 'Properties of Parallel Projections' changes positions before rasterization?
Page 29	Perspective Projections 1/3
Source cue	- Properties / - Communicate realistic views of three-dimensional objects / and give a spatial impression / - Distort objects in perspective view / - Scaling of objects is lost
Professor-style explanation	The slide 'Perspective Projections 1/3' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: - Properties / - Communicate realistic views of three-dimensional objects / and give a spatial impression / - Distort objects in perspective view / - Scaling of objects is lost. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Perspective Projections 1/3. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: - Properties / - Communicate realistic views of three-dimensional objects / and give a spatial impression / - Distort objects in perspective view / - Scaling of objects is lost. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Perspective Projections 1/3' changes positions before rasterization?
Page 30	Perspective Projections 2/3
Source cue	- Vanishing points / - For rectangular objects (e.g., axis parallel cubes) whose surface normals / are parallel to the x-, y-, or z-axis, the number of vanishing points is equal / to the number of coordinate axes intersected by the projection plane / 1-Point Perspective 2-Point Perspective 3-Point Perspective

Professor-style explanation	The slide 'Perspective Projections 2/3' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Vanishing points / - For rectangular objects (e.g., axis parallel cubes) whose surface normals / are parallel to the x-, y-, or z-axis, the number of vanishing points is equal / to the number of coordinate axes intersected by the projection plane / 1-Point Perspective 2-Point Perspective 3-Point Perspective. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Perspective Projections 2/3. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Vanishing points / - For rectangular objects (e.g., axis parallel cubes) whose surface normals / are parallel to the x-, y-, or z-axis, the number of vanishing points is equal / to the number of coordinate axes intersected by the projection plane / 1-Point Perspective 2-Point Perspective 3-Point Perspective. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Perspective Projections 2/3'?
Page 31	Perspective Projections 3/3
Source cue	- What happens if the cube is rotated so that its surface normals are not / parallel to the coordinate axes? / - Although the projection plane intersects only one coordinate axis, in this case / you get 3 vanishing points

Professor-style explanation	The slide 'Perspective Projections 3/3' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - What happens if the cube is rotated so that its surface normals are not / parallel to the coordinate axes? / - Although the projection plane intersects only one coordinate axis, in this case / you get 3 vanishing points. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Perspective Projections 3/3. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - What happens if the cube is rotated so that its surface normals are not / parallel to the coordinate axes? / - Although the projection plane intersects only one coordinate axis, in this case / you get 3 vanishing points. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Perspective Projections 3/3'?

Page 32	4.3 Camera Modeling
Source cue	Orientation of the camera and definition of the view volume
Professor-style explanation	The slide '4.3 Camera Modeling' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. The terms describe model representation before rendering: Orientation of the camera and definition of the view volume. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors.

Technical commentary	This slide is about 4.3 Camera Modeling. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. The terms describe model representation before rendering: Orientation of the camera and definition of the view volume. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation.
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Check yourself	Can you name the geometric objects and attributes represented by '4.3 Camera Modeling' before rendering begins?

Page 33	Camera Modeling
Source cue	- Application of planar geometric projections, represented by 4×4 matrices, / to project a 3D scene onto a projection screen, i.e., the canvas / - Basic camera parameters / - Camera position / - Camera orientation
Professor-style explanation	The slide 'Camera Modeling' explains how scene objects exist before they are rendered. A 3D model is not an image yet; it is a structured description of geometry and attributes. The central objects are vertices, edges, faces, triangles or other primitives, and sometimes additional data such as normals, texture coordinates, colors, materials, or connectivity. The terms describe model representation before rendering: - Application of planar geometric projections, represented by 4×4 matrices, / to project a 3D scene onto a projection screen, i.e., the canvas / - Basic camera parameters / - Camera position / - Camera orientation. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation. This matters because the rendering pipeline needs this representation as input. The model describes what exists in the scene, while later stages decide where it appears, which parts are visible, how it is sampled into fragments, and how it is shaded into final pixel colors.
Technical commentary	This slide is about Camera Modeling. The slide describes scene objects before rendering: geometric models, primitives, vertices, topology, attributes, and the representation used as input to the pipeline. The terms describe model representation before rendering: - Application of planar geometric projections, represented by 4×4 matrices, / to project a 3D scene onto a projection screen, i.e., the canvas / - Basic camera parameters / - Camera position / - Camera orientation. Geometric primitives define shape, vertices provide positions and attributes, and later pipeline stages transform, project, rasterize, and shade that representation.
Why it matters	Model slides explain the input objects that later transformations, projection, rasterization, and shading operate on.
Check yourself	Can you name the geometric objects and attributes represented by 'Camera Modeling' before rendering begins?

Page 34	View Volume
---------	-------------

Source cue	- Objects that are within the view volume are visible for the camera / - Conical viewing volume / - Clipping the scene objects against a conical viewing volume is expensive / (solving quadratic equations) / - Frustum - rectangular viewing volume
Professor-style explanation	The slide 'View Volume' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. The terms describe boundary decisions: - Objects that are within the view volume are visible for the camera / - Conical viewing volume / - Clipping the scene objects against a conical viewing volume is expensive / (solving quadratic equations) / - Frustum - rectangular viewing volume. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.
Technical commentary	This slide is about View Volume. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. The terms describe boundary decisions: - Objects that are within the view volume are visible for the camera / - Conical viewing volume / - Clipping the scene objects against a conical viewing volume is expensive / (solving quadratic equations) / - Frustum - rectangular viewing volume. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary.
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'View Volume'?
Page 35	Projection Specification
Source cue	- Specified by Height Angle, Aspect Ratio, / Near Plane Distance and Far Plane Distance / - Examples of Aspect Ratio / - NTSC 4 : 3, HDTV 16 : 9 / - Cinema 2.35 : 1, 2.39 : 1, 2.40 : 1, or 2.55 : 1

Professor-style explanation	The slide 'Projection Specification' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: - Specified by Height Angle, Aspect Ratio, / Near Plane Distance and Far Plane Distance / - Examples of Aspect Ratio / - NTSC 4 : 3, HDTV 16 : 9 / - Cinema 2.35 : 1, 2.39 : 1, 2.40 : 1, or 2.55 : 1. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Projection Specification. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: - Specified by Height Angle, Aspect Ratio, / Near Plane Distance and Far Plane Distance / - Examples of Aspect Ratio / - NTSC 4 : 3, HDTV 16 : 9 / - Cinema 2.35 : 1, 2.39 : 1, 2.40 : 1, or 2.55 : 1. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Projection Specification' changes positions before rasterization?
Page 36	View Angle
Source cue	- Parallel projection: no viewing angle as rays are parallel / - Perspective projection: / viewing angle determines perspective distortion in the image / - Specification / - Through the viewing angle in height (field of view in y, fovy)

Professor-style explanation	The slide 'View Angle' explains the problem of deciding which surface is actually visible from the current viewpoint. Many primitives can project to the same image region, but only the nearest relevant surface should determine the opaque color at a pixel. Different algorithms solve this relation in different spaces: object-space methods sort or split geometry, image-space methods subdivide regions, ray methods query visibility along viewing rays, and the depth buffer compares per-fragment depth values. The terms describe an occlusion test: - Parallel projection: no viewing angle as rays are parallel / - Perspective projection: / viewing angle determines perspective distortion in the image / - Specification / - Through the viewing angle in height (field of view in y, fovy). Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint. The object-level relation is viewpoint, candidate surface, visibility rule, and visible result. This relation is the reason that fragment generation and final pixel color are not the same thing.
Technical commentary	This slide is about View Angle. The visibility method on the slide decides which surface, fragment, ray hit, or image region is visible from the current viewpoint. The terms describe an occlusion test: - Parallel projection: no viewing angle as rays are parallel / - Perspective projection: / viewing angle determines perspective distortion in the image / - Specification / - Through the viewing angle in height (field of view in y, fovy). Candidate surfaces or ray hits are compared so the renderer can decide which object is actually visible from the current viewpoint.
Why it matters	Visibility decides which generated candidates are actually seen from the current viewpoint.
Check yourself	Can you decide whether 'View Angle' works per object, per image region, per ray, or per fragment?
Page 37	Clipping Planes
Source cue	- Position of the planes is determined by the distance along the look vector: / near distance and far distance / - Near plane hides objects that are behind or too close to the camera / - Far plane hides objects that are too far away (and may be hard to spot) / $\Rightarrow$ Reduction of rendering overhead
Professor-style explanation	The slide 'Clipping Planes' explains clipping as a geometric boundary operation. A line segment, polygon, or triangle is compared with a window, plane, or volume. The part inside the valid region survives, the part outside is removed, and a primitive crossing the boundary receives newly computed intersection points. The terms describe boundary decisions: - Position of the planes is determined by the distance along the look vector: / near distance and far distance / - Near plane hides objects that are behind or too close to the camera / - Far plane hides objects that are too far away (and may be hard to spot) / $\Rightarrow$ Reduction of rendering overhead. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary. The objects involved are the original primitive, the clipping boundary, an inside/outside classification, intersection points, and the resulting clipped primitive. The core idea is not simply deleting geometry; clipping can reshape geometry so that the rasterizer receives only the meaningful visible portion.

Technical commentary	This slide is about Clipping Planes. The clipping objects on the slide classify geometry against boundaries and produce accepted, rejected, or newly intersected primitive pieces. The terms describe boundary decisions: - Position of the planes is determined by the distance along the look vector: / near distance and far distance / - Near plane hides objects that are behind or too close to the camera / - Far plane hides objects that are too far away (and may be hard to spot) / $\Rightarrow$ Reduction of rendering overhead. A primitive is classified against a clipping region, rejected if outside, preserved if inside, or cut at intersections if it crosses a boundary.
Why it matters	Clipping decides what geometry is allowed to reach rasterization and can create new boundary vertices.
Check yourself	Can you decide what is accepted, rejected, or newly intersected in 'Clipping Planes'?

Page 38	4.4 Specifying Projections in OpenGL
Source cue	Functions to describe the view volume
Professor-style explanation	The slide '4.4 Specifying Projections in OpenGL' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: Functions to describe the view volume. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about 4.4 Specifying Projections in OpenGL. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: Functions to describe the view volume. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in '4.4 Specifying Projections in OpenGL'?
Page 39	Projection Matrix Generation 1/3
Source cue	- Convenient generation of perspective projection matrix with GLM / - <code>glm::perspective(fovy, aspect, near, far) / h -n -f / aspect ■ / fovy fovy</code>

Professor-style explanation	The slide 'Projection Matrix Generation 1/3' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Convenient generation of perspective projection matrix with GLM / - glm::perspective(fovy, aspect, near, far) / $h \cdot n \cdot f / \text{aspect}$ / fovy fovy. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Projection Matrix Generation 1/3. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Convenient generation of perspective projection matrix with GLM / - glm::perspective(fovy, aspect, near, far) / $h \cdot n \cdot f / \text{aspect}$ / fovy fovy. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Projection Matrix Generation 1/3'?

Page 40	Projection Matrix Generation 2/3
Source cue	- Alternative generation of perspective projection matrix with GLM / - glm::frustum(left, right, bottom, top, near, far) / (r, t, -n) / $z = -f$ (far plane) / (l, b, -n)
Professor-style explanation	The slide 'Projection Matrix Generation 2/3' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Alternative generation of perspective projection matrix with GLM / - glm::frustum(left, right, bottom, top, near, far) / (r, t, -n) / $z = -f$ (far plane) / (l, b, -n). Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.

Technical commentary	This slide is about Projection Matrix Generation 2/3. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Alternative generation of perspective projection matrix with GLM / - glm::frustum(left, right, bottom, top, near, far) / (r, t, -n) / z = -f (far plane) / (l, b, -n). Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Projection Matrix Generation 2/3'?

Page 41	Projection Matrix Generation 3/3
Source cue	- Implementation of glm::perspective / detail::tmat4x4<T> perspective(T const & fovy, T const & aspect, / T const & near, T const & far) { / T const & l, r, b, t; / t = n * tan(fovy * M_PI / 360.0);
Professor-style explanation	The slide 'Projection Matrix Generation 3/3' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Implementation of glm::perspective / detail::tmat4x4<T> perspective(T const & fovy, T const & aspect, / T const & near, T const & far) { / T const & l, r, b, t; / t = n * tan(fovy * M_PI / 360.0). Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Projection Matrix Generation 3/3. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Implementation of glm::perspective / detail::tmat4x4<T> perspective(T const & fovy, T const & aspect, / T const & near, T const & far) { / T const & l, r, b, t; / t = n * tan(fovy * M_PI / 360.0). Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Projection Matrix Generation 3/3'?
Page 42	Orthographic Matrix Specification

Source cue	- Convenient generation of parallel projection matrix with GLM / - <code>glm::ortho(left, right, bottom, top, near, far) / (r, t, -f) / z = -f (far plane) / z = -n (near plane)</code>
Professor-style explanation	The slide 'Orthographic Matrix Specification' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Convenient generation of parallel projection matrix with GLM / - <code>glm::ortho(left, right, bottom, top, near, far) / (r, t, -f) / z = -f (far plane) / z = -n (near plane)</code> . Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Orthographic Matrix Specification. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Convenient generation of parallel projection matrix with GLM / - <code>glm::ortho(left, right, bottom, top, near, far) / (r, t, -f) / z = -f (far plane) / z = -n (near plane)</code> . Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Orthographic Matrix Specification'?
Page 43	4.5 Orthographic Projection Derivation
Source cue	Step-wise transformation of a cuboid into a cube
Professor-style explanation	The slide '4.5 Orthographic Projection Derivation' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: Step-wise transformation of a cuboid into a cube. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.

Technical commentary	This slide is about 4.5 Orthographic Projection Derivation. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: Step-wise transformation of a cuboid into a cube. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after '4.5 Orthographic Projection Derivation'?

Page 44	Canonical View Volume
Source cue	- Canonical view volume - simplest possible view volume / - Cube centered at the origin / - Cube faces $x = \pm 1$, $y = \pm 1$, $z = \pm 1$ / - Defined in a left-handed coordinate system (LHS) / (1, 1, 1)
Professor-style explanation	The slide 'Canonical View Volume' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Canonical view volume - simplest possible view volume / - Cube centered at the origin / - Cube faces $x = \pm 1$, $y = \pm 1$, $z = \pm 1$ / - Defined in a left-handed coordinate system (LHS) / (1, 1, 1). Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Canonical View Volume. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Canonical view volume - simplest possible view volume / - Cube centered at the origin / - Cube faces $x = \pm 1$, $y = \pm 1$, $z = \pm 1$ / - Defined in a left-handed coordinate system (LHS) / (1, 1, 1). Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Canonical View Volume'?

Page 45	Orthographic Projection 1/4
Source cue	- Transform right-handed view volume specified as $[l, r] \times [b, t] \times [-n, -f]$ / into left-handed canonical view volume - $1, 1 \times -1, 1 \times -1, 1$ / and then create the projected image by neglecting z-values / (r, t, -f) / (1, 1, 1)

Professor-style explanation	The slide 'Orthographic Projection 1/4' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Transform right-handed view volume specified as $[l, r] \times [b, t] \times [-n, -f]$ / into left-handed canonical view volume $[-1, 1] \times [-1, 1] \times [-1, 1]$ / and then create the projected image by neglecting z-values / $(r, t, -f)$ / $(1, 1, 1)$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Orthographic Projection 1/4. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Transform right-handed view volume specified as $[l, r] \times [b, t] \times [-n, -f]$ / into left-handed canonical view volume $[-1, 1] \times [-1, 1] \times [-1, 1]$ / and then create the projected image by neglecting z-values / $(r, t, -f)$ / $(1, 1, 1)$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Orthographic Projection 1/4'?
Page 46	Orthographic Projection 2/4
Source cue	- Let $p(x, y, z)$ be a model point and $q(x', y', z')$ its transformed point / - The desired pixel (x, y) results from orthographic projection of q / p / onto the plane $z = -1$: $x = x'$ and $y = y' / p$

Professor-style explanation	The slide 'Orthographic Projection 2/4' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Let $p(x, y, z)$ be a model point and $q(x', y', z')$ its transformed point / - The desired pixel (x, y) results from orthographic projection of q / p onto the plane $z = -1$: $x = x'$ and $y = y' / p$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Orthographic Projection 2/4. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Let $p(x, y, z)$ be a model point and $q(x', y', z')$ its transformed point / - The desired pixel (x, y) results from orthographic projection of q / p onto the plane $z = -1$: $x = x'$ and $y = y' / p$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Orthographic Projection 2/4'?

Page 47	Orthographic Projection 3/4
Source cue	- Step 2, scale cuboid from step 1 into cube / - Original cuboid bounds are $/ r - l \ r - l \ t - b \ t - b \ f - n \ f - n / - , x - , x - , / 2 \ 2 \ 2 \ 2 \ 2 \ 2$
Professor-style explanation	The slide 'Orthographic Projection 3/4' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: - Step 2, scale cuboid from step 1 into cube / - Original cuboid bounds are $/ r - l \ r - l \ t - b \ t - b \ f - n \ f - n / - , x - , x - , / 2 \ 2 \ 2 \ 2 \ 2 \ 2$. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.

Technical commentary	This slide is about Orthographic Projection 3/4. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: - Step 2, scale cuboid from step 1 into cube / - Original cuboid bounds are $r - l, r - l, t - b, t - b, f - n, f - n$, $x - , x - , / 2$ $2 \ 2 \ 2 \ 2 \ 2 \ 2$. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Orthographic Projection 3/4' changes positions before rasterization?

Page 48	Orthographic Projection 4/4
Source cue	- Multiplication of all orthographic projection matrices results in $2 \ r + l / 0$ $0 - / r - l \ r - l / 2 \ t + b$
Professor-style explanation	The slide 'Orthographic Projection 4/4' explains how camera geometry turns a 3D scene into image coordinates. In view space, objects are described relative to the camera. The projection matrix then maps that camera-centered geometry into clip space, where the viewing volume and clipping boundaries can be handled consistently. Perspective projection uses the homogeneous coordinate so that the later divide by w creates foreshortening: farther objects occupy less image space. Orthographic projection removes that depth-based size change and keeps parallel structures visually parallel. The terms describe camera-to-screen mapping: - Multiplication of all orthographic projection matrices results in $2 \ r + l / 0 \ 0 - / r - l \ r - l / 2 \ t + b$. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport. The object chain is view-space position, projection matrix, clip coordinate, normalized device coordinate, and viewport coordinate. The slide is therefore connecting camera setup, visual appearance, and the numeric coordinate pipeline that rasterization later consumes.
Technical commentary	This slide is about Orthographic Projection 4/4. The projection objects on the slide convert view-space geometry into clip coordinates, normalized coordinates, and finally screen-related positions. The terms describe camera-to-screen mapping: - Multiplication of all orthographic projection matrices results in $2 \ r + l / 0 \ 0 - / r - l \ r - l / 2 \ t + b$. View-space geometry is mapped by a projection matrix, clipped, divided into normalized coordinates, and finally mapped to a viewport.
Why it matters	Projection controls both image composition and depth precision, so it affects visibility and rasterization later.
Check yourself	Can you explain how 'Orthographic Projection 4/4' changes positions before rasterization?

Page 49	4.6 Perspective Projection Derivation
Source cue	Step-wise transformation of a frustum into a cube

Professor-style explanation	The slide '4.6 Perspective Projection Derivation' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: Step-wise transformation of a frustum into a cube. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about 4.6 Perspective Projection Derivation. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: Step-wise transformation of a frustum into a cube. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after '4.6 Perspective Projection Derivation'?

Page 50	Perspective Projection
Source cue	- Transform view volume created by <code>glm::frustum</code> or <code>glm::perspective</code> into / canonical view volume / and then create the projected image by neglecting z-values / (1, 1, 1) / (-1,-1,-1)
Professor-style explanation	The slide 'Perspective Projection' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Transform view volume created by <code>glm::frustum</code> or <code>glm::perspective</code> into / canonical view volume / and then create the projected image by neglecting z-values / (1, 1, 1) / (-1,-1,-1). Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.

Technical commentary	This slide is about Perspective Projection. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Transform view volume created by <code>glm::frustum</code> or <code>glm::perspective</code> into / canonical view volume / and then create the projected image by neglecting z-values / (1, 1, 1) / (-1,-1,-1). Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Perspective Projection'?

Page 51	Transformation of z-Values 1/2
Source cue	- After the perspective projection, z-values shall be mapped as follows / - All z-values lying on the near plane shall have a value of -1 / - All z-values lying on the far plane shall have a value of 1 / - The following matrix P allows modification of the depth values / and assures the flip (left-handed to right-handed conversion)
Professor-style explanation	The slide 'Transformation of z-Values 1/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - After the perspective projection, z-values shall be mapped as follows / - All z-values lying on the near plane shall have a value of -1 / - All z-values lying on the far plane shall have a value of 1 / - The following matrix P allows modification of the depth values / and assures the flip (left-handed to right-handed conversion). Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Transformation of z-Values 1/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - After the perspective projection, z-values shall be mapped as follows / - All z-values lying on the near plane shall have a value of -1 / - All z-values lying on the far plane shall have a value of 1 / - The following matrix P allows modification of the depth values / and assures the flip (left-handed to right-handed conversion). Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.

Check yourself	Can you state the coordinate space before and after 'Transformation of z-Values 1/2'?
Page 52	Transformation of z-Values 2/2
Source cue	- Choosing values for a and b in $P / -a \text{ and } b \text{ must fulfill the following } / -an + b / P \cdot 0 \ 0 \ -n \ 1 \ T = 0 \ 0 \ -an + b \ n \ T \ 0 \ 0 = 0 \ 0 \ -1 / \div w \ n$
Professor-style explanation	The slide 'Transformation of z-Values 2/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Choosing values for a and b in $P / -a \text{ and } b \text{ must fulfill the following } / -an + b / P \cdot 0 \ 0 \ -n \ 1 \ T = 0 \ 0 \ -an + b \ n \ T \ 0 \ 0 = 0 \ 0 \ -1 / \div w \ n$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Transformation of z-Values 2/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Choosing values for a and b in $P / -a \text{ and } b \text{ must fulfill the following } / -an + b / P \cdot 0 \ 0 \ -n \ 1 \ T = 0 \ 0 \ -an + b \ n \ T \ 0 \ 0 = 0 \ 0 \ -1 / \div w \ n$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Transformation of z-Values 2/2'?
Page 53	Transformation of x- and y-Values 1/2
Source cue	- Derivation so far leaves us with the following matrix / which only changes z-coordinates / $1 \ 0 \ 0 \ 0 / 0 \ 1 \ 0 \ 0 / -P = f+n \ 2nf$

Professor-style explanation	The slide 'Transformation of x- and y-Values 1/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Derivation so far leaves us with the following matrix / which only changes z-coordinates / $\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}$ / $P = f+n \cdot 2nf$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Transformation of x- and y-Values 1/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Derivation so far leaves us with the following matrix / which only changes z-coordinates / $\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}$ / $P = f+n \cdot 2nf$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Transformation of x- and y-Values 1/2'?
Page 54	Transformation of x- and y-Values 2/2
Source cue	- To transform p to the coordinates (1,0,-1), the following equation must hold / $\alpha \tan / \alpha n \cdot \tan / n \cdot \tan 2 / 2 \cdot 1$
Professor-style explanation	The slide 'Transformation of x- and y-Values 2/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - To transform p to the coordinates (1,0,-1), the following equation must hold / $\alpha \tan / \alpha n \cdot \tan / n \cdot \tan 2 / 2 \cdot 1$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.

Technical commentary	This slide is about Transformation of x- and y-Values 2/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - To transform p to the coordinates (1,0,-1), the following equation must hold / $\alpha \tan / \alpha n \cdot \tan / n \cdot \tan 2 / 2 2 1$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Transformation of x- and y-Values 2/2'?

Page 55	4.7 Viewport Transformation
Source cue	Mapping the normalized device coordinates to the screen
Professor-style explanation	The slide '4.7 Viewport Transformation' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: Mapping the normalized device coordinates to the screen. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about 4.7 Viewport Transformation. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: Mapping the normalized device coordinates to the screen. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after '4.7 Viewport Transformation'?

Page 56	Screen Coordinate Mapping
Source cue	- Mapping of the viewing plane on a drawing surface / - Define the pixel rectangle to be mapped to / - Initially, this rectangle is the size of the entire canvas / - <code>glViewport(GLint x, GLint y, GLsizei width, GLsizei height)</code> / - When window size changes, adjust the aspect ratio to avoid distortion

Professor-style explanation	The slide 'Screen Coordinate Mapping' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Mapping of the viewing plane on a drawing surface - Define the pixel rectangle to be mapped to - Initially, this rectangle is the size of the entire canvas - <code>glViewport(GLint x, GLint y, GLsizei width, GLsizei height)</code> - When window size changes, adjust the aspect ratio to avoid distortion. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Screen Coordinate Mapping. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Mapping of the viewing plane on a drawing surface - Define the pixel rectangle to be mapped to - Initially, this rectangle is the size of the entire canvas - <code>glViewport(GLint x, GLint y, GLsizei width, GLsizei height)</code> - When window size changes, adjust the aspect ratio to avoid distortion. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Screen Coordinate Mapping'?
Page 57	Viewport Transformation 1/2
Source cue	- Specification of the viewport parameters by <code>glm::perspective</code> / Projection / <code>glm::frustum</code> / transformation / - <code>glViewport(x, y, w, h)</code> <code>glm::ortho</code>

Professor-style explanation	The slide 'Viewport Transformation 1/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Specification of the viewport parameters by glm::perspective / Projection / glm::frustum / transformation / - glViewport(x, y, w, h) glm::ortho. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.
Technical commentary	This slide is about Viewport Transformation 1/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Specification of the viewport parameters by glm::perspective / Projection / glm::frustum / transformation / - glViewport(x, y, w, h) glm::ortho. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Viewport Transformation 1/2'?
Page 58	Viewport Transformation 2/2
Source cue	- Viewport transformation is a 2D transformation defined as $p = M \cdot p$ with $s \cdot N / \Delta x \Delta y / - M = T \cdot x, y \cdot S \cdot S, S \cdot T -x, -y / s$ Smin Smin Nmin Nmin
Professor-style explanation	The slide 'Viewport Transformation 2/2' explains how geometric objects move through coordinate systems. A point has a location, a vector has a direction and magnitude, a normal describes surface orientation, and a coordinate frame defines how these quantities are measured. A transformation matrix changes the description of these objects: translation moves points, rotation changes orientation, scaling changes size, and composition combines several operations into one matrix product. The terms describe a coordinate-space conversion: - Viewport transformation is a 2D transformation defined as $p = M \cdot p$ with $s \cdot N / \Delta x \Delta y / - M = T \cdot x, y \cdot S \cdot S, S \cdot T -x, -y / s$ Smin Smin Nmin Nmin. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space. The central relation is source space, transformation, and target space. In computer graphics this relation is everywhere: object space becomes world space, world space becomes view space, and view space becomes clip space. A formula is only meaningful after the two coordinate spaces around it are clear.

Technical commentary	This slide is about Viewport Transformation 2/2. The transformation objects on the slide move points, vectors, or coordinate frames from one space into another using matrices or affine operations. The terms describe a coordinate-space conversion: - Viewport transformation is a 2D transformation defined as $p = M \cdot p$ with $/s \leq N / \Delta x \Delta y / - M = T x, y \cdot S S, S \cdot T -x, -y / s$ $S_{min} S_{min} N_{min} N_{min}$. Input positions or vectors are transformed by matrices so the same object can be expressed in model, world, view, or clip space.
Why it matters	A wrong coordinate-space assumption can make correct formulas produce wrong images.
Check yourself	Can you state the coordinate space before and after 'Viewport Transformation 2/2'?

Page 59	volume, we can differentiate different projection types
Source cue	- Projection has its origins in the arts / - Based on the deformation of the initial view volume to the canonical view / volume, we can differentiate different projection types / - In OpenGL two main projection types exist / - Perspective projection results in perspective shortening
Professor-style explanation	The slide 'volume, we can differentiate different projection types' explains how OpenGL exposes the rendering pipeline through explicit objects and state. A context owns the current rendering state; buffers hold vertex, index, texture, or pixel data; shader programs define programmable processing; textures and samplers provide sampled data; framebuffers receive the result. OpenGL does not render from intention, it renders from the objects that are bound and the state that is active at the moment of the draw call. The terms connect CPU-side API control with GPU execution: - Projection has its origins in the arts / - Based on the deformation of the initial view volume to the canonical view / volume, we can differentiate different projection types / - In OpenGL two main projection types exist / - Perspective projection results in perspective shortening. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written. The concrete relation is API command, GPU resource, shader interface, and visible result. A small mismatch in this relation, such as a wrong buffer layout, missing uniform, wrong texture unit, or disabled depth test, can produce a perfectly valid draw call with a completely wrong image.
Technical commentary	This slide is about volume, we can differentiate different projection types. The OpenGL objects and calls shown here control GPU state, bound resources, shader interfaces, buffer contents, or framebuffer access at draw time. The terms connect CPU-side API control with GPU execution: - Projection has its origins in the arts / - Based on the deformation of the initial view volume to the canonical view / volume, we can differentiate different projection types / - In OpenGL two main projection types exist / - Perspective projection results in perspective shortening. Objects, bindings, shader interfaces, buffers, and framebuffer state determine what data is processed and where results are written.
Why it matters	OpenGL bugs are usually state, binding, shader-interface, or buffer-layout bugs, so API details matter.
Check yourself	Can you name the OpenGL object, state, shader stage, or buffer involved in 'volume, we can differentiate different projection types'?

Page 60	Literature and other sources used in this chapter
Source cue	No object-level text was extracted from this page; the page is primarily title, image, diagram, or layout content.
Professor-style explanation	The slide 'Literature and other sources used in this chapter' is primarily a visual, title, transition, or diagram page rather than a text-heavy concept slide. Its role is to place the next concept into the lecture flow and give a visual anchor for the topic that follows. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text. In the surrounding lecture sequence, this kind of page usually marks a shift of attention: from one pipeline stage to another, from a general idea to an algorithm, or from theory to an implementation detail.
Technical commentary	This slide is about Literature and other sources used in this chapter. The slide lists source material or chapter references that support the technical content and give names for further reading. The extracted text does not expose object-level labels for this page. Within the lecture flow, the page functions as a visual, diagram, or title transition, so its connection is established by the surrounding slides rather than by readable bullet text.
Why it matters	Reference slides provide the source trail for definitions, algorithms, and deeper explanations.
Check yourself	Can you identify which source or topic 'Literature and other sources used in this chapter' points to for deeper study?
Page 61	- Text Books
Source cue	- J. Foley, A. van Dam, S. Feiner: Computer Graphics: Principles and Practice (3rd / - P. Shirley, M. Ashikhmin, S. Marschner: Fundamentals of Computer Graphics
Professor-style explanation	The slide '- Text Books' introduces a graphics object, operation, or relation that belongs to the rendering workflow. The named terms describe what data enters the step, what operation changes or classifies that data, and what result is passed onward. The terms form an input-operation-output relation: - J. Foley, A. van Dam, S. Feiner: Computer Graphics: Principles and Practice (3rd / - P. Shirley, M. Ashikhmin, S. Marschner: Fundamentals of Computer Graphics. The slide groups them because one object or quantity is processed into another result used later in rendering. The object-level relation is input, operation, output, and later use. This is the basic shape of most computer graphics concepts: data is represented in one form, processed by a rule or algorithm, and then consumed by the next stage of image generation.
Technical commentary	This slide is about - Text Books. The slide names a concrete relation between input data, an operation, and an output that another graphics stage can consume. The terms form an input-operation-output relation: - J. Foley, A. van Dam, S. Feiner: Computer Graphics: Principles and Practice (3rd / - P. Shirley, M. Ashikhmin, S. Marschner: Fundamentals of Computer Graphics. The slide groups them because one object or quantity is processed into another result used later in rendering.
Why it matters	Even a sparse slide usually names a relation you must be able to explain in words.

04.1 Linear Perspective and Planar Projections

Source location: Sections 4.1-4.2

Commentary

Perspective projection models the visual effect that farther objects appear smaller. Orthographic projection removes this distance-based size change and keeps parallel lines parallel. Both are useful, but they communicate different spatial relationships.

Planar projection means mapping points onto an image plane. The lecture's art examples are not decoration; they show that projection is a geometric rule for constructing an image from a viewpoint.

Mental model

Projection is the rule that turns 3D positions into image-plane positions.

Check yourself

What visual cue does perspective projection add that orthographic projection lacks?

04.2 Camera Modeling

Source location: Section 4.3

Commentary

A virtual camera is defined by position, orientation, projection type, viewing volume, and image/viewport settings. The view transform places the world relative to the camera. The projection transform maps that view into clip coordinates.

Near and far clipping planes are part of the camera model. They decide what range of depths is represented and strongly affect depth buffer precision. A careless near/far setup can create z-fighting even if the scene geometry is correct.

Mental model

A camera is not only an eye position; it is also a volume and a mapping rule.

Check yourself

Why can changing the near plane affect depth artifacts?

04.3 Specifying Projections in OpenGL

Source location: Section 4.4

Commentary

In OpenGL, projection is usually encoded in a projection matrix sent to a shader. The matrix maps view-space coordinates to clip space. Clipping and the perspective divide then lead toward normalized device coordinates.

Aspect ratio and field of view must match the intended viewport. If the aspect ratio is wrong, objects appear stretched. If the field of view is extreme, the image can look distorted even though the math is functioning.

Mental model

The projection matrix is the camera lens encoded as linear algebra in homogeneous coordinates.

Check yourself

What symptom suggests that the projection aspect ratio does not match the window?

04.4 Orthographic and Perspective Derivations

Source location: Sections 4.5-4.6

Commentary

The derivations are there to explain where the matrix entries come from. Orthographic projection maps a box-shaped view volume into normalized coordinates. Perspective projection maps a frustum and uses the homogeneous component for the later divide.

You do not need to treat these matrices as random formulas. Each part maps a coordinate range into a standard range. The perspective matrix additionally arranges w so that the perspective divide creates foreshortening.

Mental model

Projection matrices normalize a viewing volume; perspective also prepares the divide by w .

Check yourself

What does the perspective divide do to x and y coordinates?

04.5 Viewport Transformation

Source location: Section 4.7

Commentary

After normalized device coordinates, the viewport transform maps the normalized range to actual window coordinates. This is the last geometric mapping before rasterization uses the target grid.

Separating projection from viewport mapping helps debugging. A wrong projection can produce wrong normalized coordinates; a wrong viewport can map otherwise valid coordinates into the wrong part of the window.

Mental model

Projection decides normalized position; viewport decides where that position lands on the screen.

Check yourself

Why is resizing a window related to both viewport and projection settings?

End-of-lecture summary

If you remember only one thing from Lecture 04, remember this: This lecture explains how a 3D view becomes a 2D image. Projection is where camera geometry, homogeneous coordinates, clipping planes, depth precision, and viewport mapping meet.

Before moving on, you should be able to explain the lecture without reading slide bullets aloud. Use the vocabulary from the slides, but add causal links: why a step exists, what data it consumes, what it produces, and what can go wrong.